

Cook's Theorem

In 1971 Stephen Cook proved a groundbreaking result in complexity theory, namely that SAT is an NP-complete decision problem. We dedicate this lecture to its proof.

Cook's Theorem. SAT is $\mathcal{NP}$ -complete.

The proof of Cook's Theorem involves showing a polynomial-time reduction from any language $L \in \mathcal{NP}$ to the SAT language. To start off, let

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$$

be an NTM such that $L(M) = L$, and assume that each branch of the computation of M on input w is bounded by polynomial $p(n)$ where $n = |w|$. To simplify matters, we assume that each branch of $T(M, w)$ has *exactly* $p(n)$ configuration nodes. The easiest way to achieve this is to assume that, if a node labeled with a final configuration κ occurs on a branch at a depth less than $p(n)$, then that node will have one child which is also labeled with a final configuration node of the same type (e.g. accepting begets accepting). The new configuration will have the same state, but perhaps M does something trivial, such as moves right without writing on the tape.

The main idea behind the reduction is to map NTM M and arbitrary Turing machine input w to a Boolean formula $F(M, w)$ whose satisfiability is equivalent to the existence of some computation branch of $T(M, w)$ that is accepting. The clever idea in the proof is that the reduction algorithm does not bother to understand the steps of each of the branches of $T(M, w)$ (indeed, this would take longer than polynomial time for most NTM's). Instead it produces families of Boolean formulas that allow for the simulation of any branch of $T(M, w)$ via an appropriate truth assignment for each of the variables of $F(M, w)$. Thus the “complexity and structure” of $T(M, w)$ remains after the reduction, and that complexity is witnessed by the exponentially large number of possible truth assignments that yield the different simulations of the different branches of $T(M, w)$.

Defining the variables of $F(M, w)$. We need enough variables to keep track of the computation along any one branch of the tree $T(M, w)$. Notice that, since M makes no more than $p(n)$ head moves on any branch, we may assume that the tape head will never reach beyond cell $p(n)$. Thus, the following four variable families will suffice.

1. **Current state of the machine.** For each $q \in Q$ and for each $0 \leq j \leq p(n)$, $s(q, j)$ is the Boolean variable whose truth is equivalent to the statement that “ M is in state q during the j th step of the computation branch”. Which computation branch? One that hopefully ends in an accepting configuration! The beauty of the reduction is that to perform it one does not need to possess global information about any one branch of the computation. Rather the global information unfolds after the reduction and when attempting to find a satisfying assignment for $F(M, w)$ which is equivalent to searching for an accepting branch of the computation of M on input w .
2. **Location of the tape head.** For each $0 \leq i, j \leq p(n)$, l_{ij} is true iff the tape head is located at cell i during step j of the computation.
3. **Contents of each cell.** For each $t \in \Gamma$ and for each $0 \leq i, j \leq p(n)$, $c(i, j, t)$ is true iff cell i contains symbol t during step j of the computation.
4. **Unchanging of a cell.** for every $0 \leq i \leq p(n)$ and for every $0 \leq j \leq p(n) - 1$, $u(i, j)$ is true iff cell i is unchanged from step j to step $j + 1$ of the computation.

Using the variables defined above we now set out to define several families of Boolean formulas which, when ANDed together, yield $F(M, w)$. Each family serves the purpose of modeling a statement about Turing machine computations, whether it be about i) a general rule involving a Turing machine computation, ii) the specific rules involving the computation of M on input w , or iii) the logical relationship between one or more of the variable families.

Statement 1. A Turing machine is in exactly one state during any step of the computation. For every $0 \leq j \leq p(n)$,

$$\bigvee_{q \in Q} s(q, j),$$

and for every $q \in Q$,

$$s(q, j) \rightarrow \bigwedge_{\hat{q} \in Q - \{q\}} \overline{s(\hat{q}, j)}.$$

Statement 2. A Turing machine tape head is in exactly one location during any step of the computation. For every $0 \leq j \leq p(n)$,

$$\bigvee_{i=0}^{p(n)} l_{ij},$$

and for every $0 \leq k \leq p(n)$,

$$l_{kj} \rightarrow \bigwedge_{i \neq k} \overline{l_{ij}}.$$

Statement 3. Each tape cell of Turing machine M contains exactly one symbol from Γ during any step of the computation. For every $0 \leq i, j \leq p(n)$,

$$\bigvee_{t \in \Gamma} c(i, j, t),$$

and for every $t \in \Gamma$,

$$c(i, j, t) \rightarrow \bigwedge_{\hat{t} \in \Gamma - \{t\}} \overline{c(i, j, \hat{t})}.$$

Statement 4. If a tape cell remains unchanged from one step to the next, then the cell must contain the same symbol from one step to the next. For every $0 \leq i \leq p(n)$, for every $0 \leq j \leq p(n) - 1$, and for every $t \in \Gamma$,

$$c(i, j, t) \wedge u(i, j) \rightarrow c(i, j + 1, t).$$

Statement 5. The initial configuration of the computation of M on input w begins in state q_0 , with the tape head located at cell 0, and with the first $p(n) + 1$ tape cells reading $w(\sqcup)^{p(n)+1-n}$. This family has just one formula!

$$s(q_0, 0) \wedge l_{00} \wedge c(0, 0, w_1) \wedge \cdots \wedge c(n - 1, 0, w_n) \wedge c(n, 0, \sqcup) \wedge \cdots \wedge c(p(n), 0, \sqcup).$$

Statement 6. Given a Turing machine configuration, then next configuration is determined by application of the δ transition function. For every $0 \leq i \leq p(n)$, for every $0 \leq j \leq p(n) - 1$, for every $t \in \Gamma$, and for every $q \in Q$, let

$$\delta(q, t) = \{(q_1, t_1, d_1), \dots, (q_m, t_m, d_m)\}.$$

Then

$$[s(q, j) \wedge l_{ij} \wedge c(i, j, t)] \rightarrow \bigwedge_{k \neq i} u(k, j) \wedge [(s(q_1, j+1) \wedge l_{\phi(1)(j+1)} \wedge c(i, j+1, t_1)) \vee \cdots \vee (s(q_m, j+1) \wedge l_{\phi(m)(j+1)} \wedge c(i, j+1, t_m))],$$

where, for $1 \leq k \leq m$,

$$\phi(k) = \begin{cases} i - 1 & \text{if } d_k = L \text{ and } i > 0 \\ i & \text{if } d_k = L \text{ and } i = 0 \text{ or } d_k = R \text{ and } i = p(n) \\ i + 1 & \text{if } d_k = R \text{ and } i < p(n) \end{cases}$$

Statement 7. The final configuration of a computation branch of $T(M, w)$ is accepting. Recall that we want $F(M, w)$ to be satisfiable if and only if some computation branch of $T(M, w)$ is accepting. This family also has just one formula, but it is pivotal for ensuring that satisfiability will be equivalent with acceptance. That formula is simply $s(q_a, p(n))$.

Lemma 1. Let $F(M, w)$ be the AND of all each formula in each of the above seven Boolean formula families. Then $F(M, w)$ is effectively computable given M and w , and $|F(M, w)|$ is bounded by a polynomial in the length n of w .

Proof of Lemma 1. The effective computability of $F(M, w)$ follows from the fact that each Boolean formula from each family can be written effectively given each of the parameters that it depends upon. As for $|F(M, w)|$, a study of each formula family and the number of formulas in each family finds that Family 6 yields the asymptotically fastest growing family at $O(|Q||\Gamma|(p(n)^3))$ which is a polynomial. QED

Lemma 2. Let $F(M, w)$ be as defined in Lemma 1. Then $F(M, w)$ is satisfiable if and only if $T(M, w)$ has at least one accepting computation branch.

Proof of Lemma 2. Assume $T(M, w)$ has an accepting computation branch. By following the computation along this branch, one can obtain all the truth-assignment values for an assignment a that satisfies $F(M, w)$. This is true since the following information can be recorded for such a branch at each step of the computation: the current state, the current location of the tape head, the contents of each tape cell, and which of the cells will remain unchanged going to the next step of the computation. For example, if the machine is in state q at step 10 of the computation, then we know to assign $s(q, 10)$ the value of 1, and $s(\hat{q}, 10)$ the value of 0, for all $\hat{q} \in Q - \{q\}$. As another example, if we observe that the contents of cell 10 did not change from step 8 to step 9 of the computation, then we assign $u(10, 8)$ the value 1. In other words, we have designed $F(M, w)$ such that, at the very least, the variable assignments must at least represent a legal computation branch of $T(M, w)$. Granted such an assignment, the satisfiability of $F(M, w)$ will then reduce to whether or not any computation branch is accepting, since this determines whether or not the final formula $s(q_a, p(n))$ is satisfied. QED

Cook's Theorem actually gives us more than we set out to prove. We say a formula F is a *Conjunctive Normal Form (CNF)* formula iff it can be written as a conjunction of clauses, where each clause is a disjunction of literals (a literal is a variable or its negation). Moreover SAT_{CNF} is the language of all encodings of CNF formulas that are satisfiable.

Theorem 2. SAT_{CNF} is $\mathcal{NP}$ -complete.

Proof of Theorem 2. In the reduction from an arbitrary $\mathcal{NP}$ language to SAT, we constructed the formula $F(M, w)$ which is the conjunction of family members from each of the seven formula families. It is left as an exercise to show that each member of each family member is either a CNF formula, or can easily be converted one using the rules of logic. Finally, noting that the conjunction of CNF formulas is itself a CNF formula, the statement is proved. QED